VIVEKANAND EDUCATION SOCIETY
INSTITUTE OF TECHNOLOGY (AUTONOMOUS)

(Affiliated to University of Mumbai, Approved by AICTE & Recognized by Govt. of Maharashtra)

# Smart Energy Grid Optimization Using Reinforcement Learning

Submitted in fulfilment of the requirements of the

Mini Project in

Reinforcement Learning Subject

by

Mohammed Arshad Shaikh (D16ADB / 49)

**Department of Artificial Intelligence and Data Science**

**Vivekanand Education Society's Institute of Technology**

**2025-2026**

# 1. Abstract

This project presents a smart energy grid optimization system using Reinforcement Learning, specifically the Deep Q-Network (DQN) algorithm. The system simulates a microgrid environment consisting of solar and wind energy sources, battery storage, dynamic electricity pricing, and consumer demand patterns. The objective is to intelligently manage energy flow—deciding when to store, use, buy, or sell energy—to minimize cost and maximize renewable energy utilization.

A DQN agent is trained to learn optimal decision-making policies by interacting with the environment. The system includes a real-time dashboard for visualization, a Flask-based backend API, and a simulation environment built using Gymnasium. The results demonstrate improved energy efficiency, reduced grid dependency, and intelligent battery management.

# 2. Introduction

With increasing energy demand and climate concerns, integrating renewable energy sources into power grids has become essential. However, renewable sources like solar and wind are intermittent and require intelligent management.

This project aims to solve this problem using Reinforcement Learning (RL), where an agent learns to optimize energy usage in a microgrid environment. The system simulates real-world energy patterns and uses a Deep Q-Network (DQN) to make decisions dynamically.

# 3. Problem Statement

Traditional energy systems lack adaptability to fluctuating renewable sources and dynamic pricing. The challenge is:

- ➔ Efficiently manage renewable energy sources
- ➔ Minimize dependency on expensive grid energy
- ➔ Optimize battery usage
- ➔ Avoid energy wastage and blackouts

# 4. Objectives

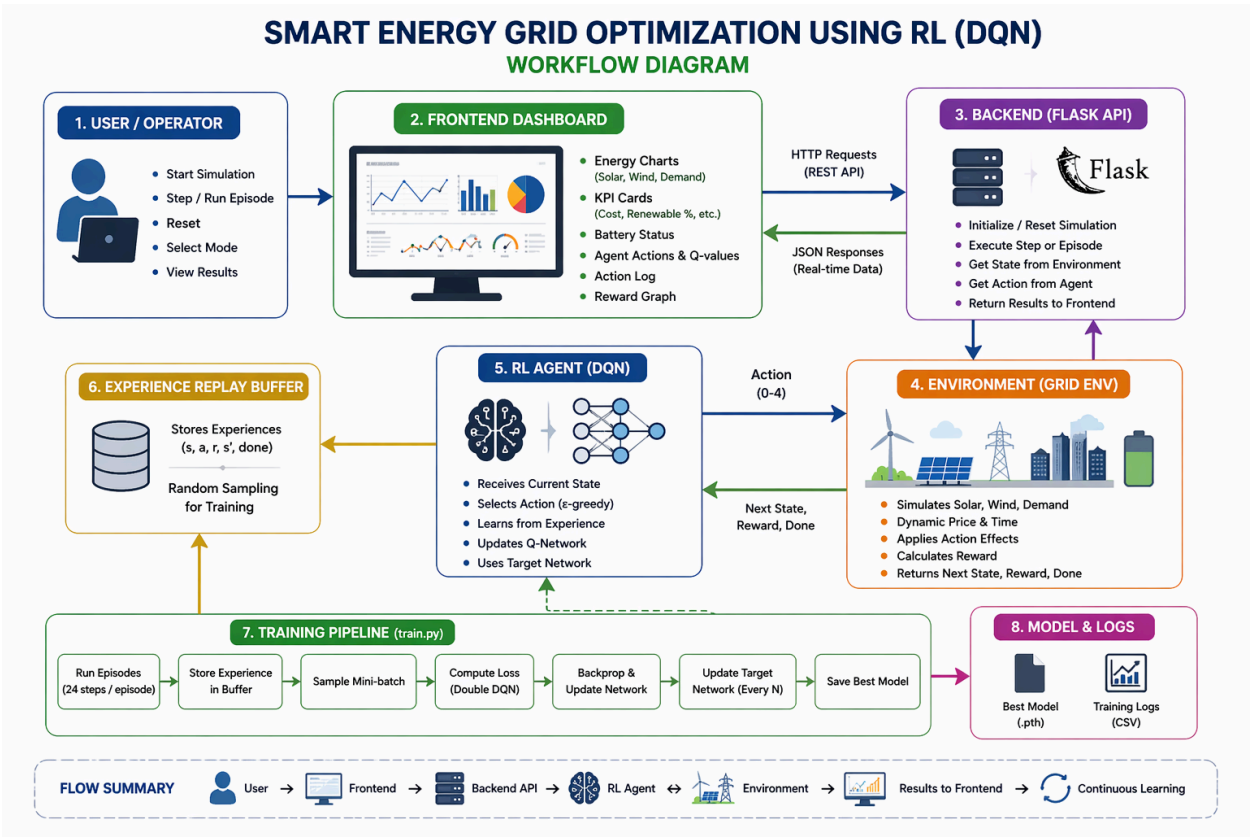
- ➔ Develop a smart microgrid simulation environment
- ➔ Implement a Deep Q-Network (DQN) agent
- ➔ Optimize energy usage using reinforcement learning
- ➔ Minimize operational cost
- ➔ Maximize renewable energy utilization
- ➔ Provide a real-time visualization dashboard

# 5. System Architecture

## Components:

1. **Frontend (Dashboard)** – Visualization using HTML, CSS, Chart.js
2. **Backend (Flask API)** – Handles simulation and communication
3. **Environment (Gym-based)** – Simulates grid behavior
4. **DQN Agent** – Learns optimal policies
5. **Training Pipeline** – Trains and saves the model

## Workflow:




# 6. Methodology

## 6.1 Reinforcement Learning Overview

Reinforcement Learning is a trial-and-error learning approach where an agent interacts with an environment and learns from rewards.

## 6.2 State Space

The state consists of 8 features:

- ➔ Solar output
- ➔ Wind output
- ➔ Demand
- ➔ Battery state of charge
- ➔ Grid price
- ➔ Time (sin)
- ➔ Time (cos)
- ➔ Day of week

## 6.3 Action Space

Discrete actions:

1. Idle
2. Charge battery
3. Discharge battery
4. Buy from grid
5. Sell to grid

## 6.4 Reward Function

The reward function is designed as:

- ➔ Positive reward for using renewable energy
- ➔ Negative reward for buying expensive grid energy
- ➔ Penalty for energy blackout
- ➔ Reward for selling energy
- ➔ Penalty for battery imbalance

## 6.5 DQN Algorithm

The Q-learning update:

$$Q(s,a) = Q(s,a) + \alpha[r + \gamma a' max Q(s',a') - Q(s,a)]$$

- ➔ $\gamma = 0.95$ (discount factor)
- ➔ $\varepsilon$-greedy strategy used for exploration

# 7. Implementation

## 7.1 Technologies Used

- ➔ Python
- ➔ Flask
- ➔ PyTorch
- ➔ Gymnasium
- ➔ NumPy
- ➔ HTML/CSS/JavaScript
- ➔ Chart.js

## 7.2 Modules

**1. Environment (grid_env.py)**

- ➔ Simulates solar, wind, demand, pricing
- ➔ Implements step() function for RL interaction

**2. Agent (dqn_agent.py)**

- ➔ Neural network with 3 hidden layers
- ➔ Uses Double DQN
- ➔ Implements epsilon-greedy policy

**3. Backend (app.py)**

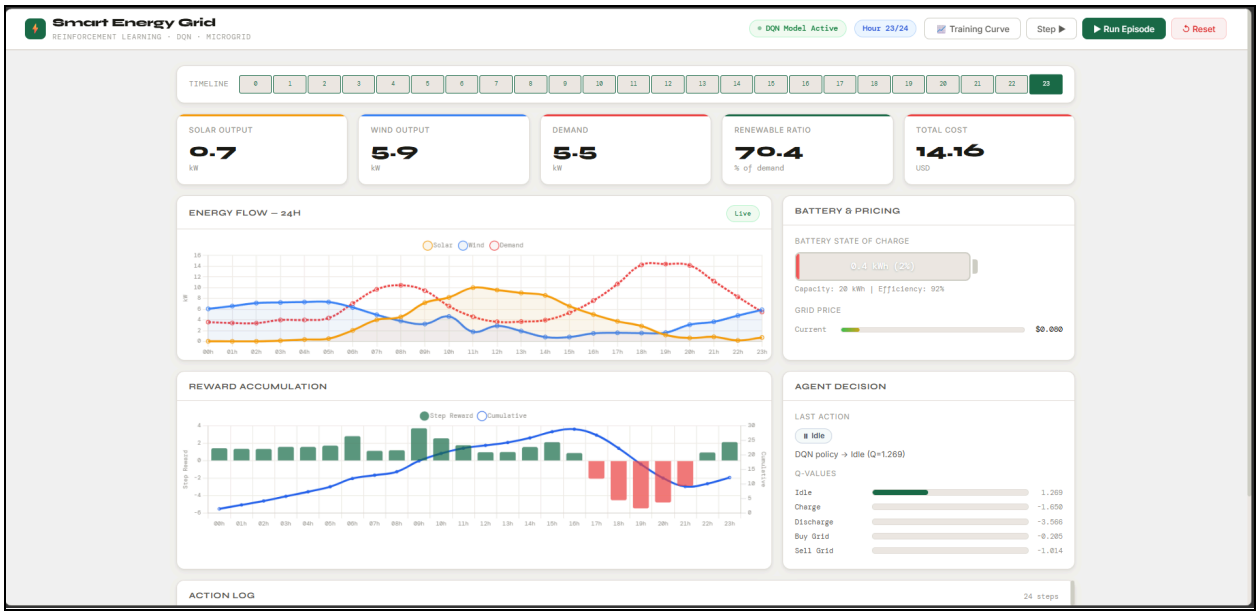
- ➔ Provides REST APIs
- ➔ Handles simulation control
- ➔ Streams real-time data

**4. Training (train.py)**

- ➔ Runs episodes
- ➔ Logs performance
- ➔ Saves best model

**5. Frontend (index.html)**

- ➔ Displays charts and KPIs
- ➔ Shows agent decisions and Q-values




# 8. Results and Analysis

- ➔ The agent learns to:
  - ◆ Store energy during surplus
  - ◆ Use battery during peak pricing
  - ◆ Sell energy when profitable
- ➔ Reduced cost compared to the heuristic policy
- ➔ Improved renewable energy utilization
- ➔ Stable learning achieved using Double DQN

# 9. Advantages

- ➔ Intelligent energy optimization
- ➔ Real-time visualization
- ➔ Scalable and modular design
- ➔ Reduces energy cost and wastage
- ➔ Demonstrates real-world applicability

# 10. Limitations

- ➔ Simulated environment (not real-world data)
- ➔ Requires training time
- ➔ Performance depends on reward design
- ➔ Limited to a single microgrid scenario

## 11. Future Scope

- ➔ Integration with real-time IoT data
- ➔ Multi-agent grid systems
- ➔ Use of advanced RL (PPO, SAC)
- ➔ Smart city energy management
- ➔ Deployment in real-world grids

## 12. Conclusion

This project successfully demonstrates the application of Reinforcement Learning in smart energy management. The DQN agent learns efficient policies to balance energy usage, reduce costs, and maximize renewable energy utilization. The system showcases how AI can contribute to sustainable and intelligent energy systems.